

CPSC 335, Algorithm Engineering – Spring 2026

Project 3 Part A: Selection ^{vs} Merge ^{vs} Quick Sort

Student Learning Objectives:

Demonstrate your ability to

- Translate descriptions of algorithms into pseudocode
- Analyze your pseudocode mathematically
- Implement your algorithm in C++
- Test your implementation
- Describe your results
- Observe that timings correspond to Big-O trends
- Translate requirements into solutions in a timely manner

In this project you will implement and compare four algorithms that solve the same problem. The first is a greedy optimized selection sort, while the second and third are a divide-n-conquer merge sort, and the fourth is a Las Vegas randomization quick sort. All four algorithms solve the problem of sorting strings in non-descending order. The goal is to compare the performance impact of changing to an algorithm with a theoretically better efficiency class versus changing to an algorithm with the same efficiency class but different constant factors. This experiment will test the following hypotheses:

1. Sorting algorithms that operate in-place on vectors are faster, by constant factors, than sorting algorithms that operate on linked-lists and/or out-of-place.
2. $O(n \log_2 n)$ -time algorithms outperform $O(n^2)$ -time algorithms for large n , regardless of the implementation choices that are made.

Description - The Word Sorting Problem:

Our theory predicts that the

- $O(n \log_2 n)$ -time sorting algorithms outperform $O(n^2)$ -time algorithms for large n , regardless of the implementation choices that are made
- Sorting algorithms that operate in-place on vectors are faster, by constant factors, than sorting algorithms that operate on linked-lists and/or out-of-place

Your experiment will show whether this is the case.

The word sorting problem is:

Input:

1. A collection C of $n \geq 2$ sanitized words w consisting of non-whitespace characters c

Output:

1. C modified such that $w_i \leq w_{i+1}$ for $0 \leq i < n - 1$

Algorithm 1: In-Place Selection Sort

The first algorithm is the *In-Place Selection Sort*, as described in ADITA 0.9.12 section 5.6. Selection sort takes $O(n^2)$ -time, but it is very simple and avoids expensive operations such as memory allocation, recursion, and generating random numbers. Therefore, we expect it to have the best constant factors of all three algorithms. Hypothesis 1 suggests that selection sort will be fastest for very small n , perhaps $n \leq 100$. However our theory predicts that the other $O(n \log_2 n)$ -time algorithms will be faster for large n .

Algorithm 2: Merge Sort on Vectors

The second algorithm is the *Merge Sort* algorithm, as described in ADITA 0.9.12 section 8.4, using a vector data structure as input and output. Merge sort takes $O(n \log_2 n)$ -time, so we expect this algorithm's runtime to be proportional to that of the highly optimized `std::sort(...)` provided by the C++ Standard Template Library (STL). However, merge sort is an out-of-place algorithm meaning it allocates and frees memory, which is a relatively slow $O(1)$ -time operation. It is also recursive, and recursion tends to have worse constant factors than iterative loops. We therefore expect our vector merge sort to be faster than our selection sort for large n , but slower than `std::sort(...)` by a constant factor.

Algorithm 3: Merge Sort on Linked Lists

The third algorithm is the *Merge Sort* algorithm again but this time using a doubly linked list data structure as input and output. The list merge sort should still take $O(n \log_2 n)$ -time, so like vector merge sort, we expect it to be faster than selection sort for large n but slower than `std::sort(...)` by a constant factor. As argued in ADITA 0.9.12 section 4.5, linked lists tend to be slower than vectors by a constant factor, since lists have poor locality of reference and therefore trigger more cache misses. So we expect list merge sort to have the slowest constant factor of the three $O(n \log n)$ -time sorts.

Algorithm 4: Quick Sort

The fourth algorithm is the In-Place *Quick Sort* algorithm as described in ADITA 0.9.12 section 9.6. Quick sort takes $O(n \log_2 n)$ -expected time but $O(n^2)$ -worst case time. By generating a uniformly distributed pseudo-random number to randomly select the pivot point we effectively gamble with the resources (time and space) consumed to guide decisions that impact the efficiency of the algorithm, but not its correctness, which is the Las Vegas pattern. The Monte Carlo algorithm, by contrast, gambles with the correctness of its output.

What to Do

You may work in groups of up to 2 people, but you must deliver individually. The Project Report must be individually handwritten by each group member (No, you can't deliver your groupmates' handwritten report as your own.)

Project Report: Produce and deliver a brief **handwritten** report scanned into **PDF format**. Your handwriting must be absolutely and perfectly legible to earn credit. Treat this not as work in progress, but rather your final, polished, college-level quality delivery to your customer. Submit your PDF file to Canvas. You can submit just once. Part B becomes available only after you submit Part A. Your Part B's algorithm's implementation must match your pseudo code as presented in this report, so get your pseudocode right before attempting to implement. Your report must include the following:

1. Algorithm 1: In-Place Selection Sort
 - a. Design a in-place selection sort algorithm for the above problem. Write detailed, clear, countable, correct, Python-like pseudocode.
 - b. Compute the step count and provide your Time Estimation function $T(n) = \dots$
 - i. Label each line of pseudocode in the left margin with its step count. Annotate the scope of any loops.
Pro Tip: Draw a faint vertical line down the page to help reserve space for your anotations
 - ii. Harvest those step counts into open-form notation (i.e., show your summations $T(n) = \sum(\dots)$)
 - iii. Show your work as you reduce that into closed-form notation citing the reduction formulas used from ADITA sections 5.2
 - iv. Evaluate your function for 3 unique values for n showing the resulting step count
 - c. Analyze your algorithm mathematically. Write a detailed, step-by-step proof proving a specific Big-O efficiency class. Clearly identify from section 3.7 of ADITA which proof methodology you are using.
2. Algorithm 2: Merge Sort on Vectors
 - i. Same as above, but for the Merge Sort on Vectors pattern
3. Algorithm 3: Merge Sort on Linked Lists
 - i. Same as above, but for the Merge Sort on Linked Lists pattern
4. Algorithm 4: Quick Sort
 - i. Same as above, but for the Quick Sort pattern